

Claude Code

슬래시 커맨드 완전정복 중요도(상중하)분류로 익히는 실전가이드



01 슬래시 커맨드 중요도 분류 기준 목록

학습순서: 상급 → 중급 → 하급(필요할 때)

상급	/clear, /compact, /context, /plan, /copy, /export, /security-review 결과물 품질에 직결하는 최우선 핵심 커맨드
중급	/resume, /rewind, /fork, /btw, /add-dir, /tasks, /desktop, /remote-control 일상적으로 사용하면 작업 효율이 향상되는 커맨드
하급	/debug, /privacy-settings, /sandbox, /model, /mcp, /hooks, /config ... (20+) 알아두면 유용하거나 UI 설정으로 대체 가능한 항목
분류분석	/usage, /stats, /insights 사용 현황 파악 및 워크플로우 개선 모니터링

02 상승: 결과물 품질에 직결하는 7대 핵심 커맨드

/clear

컨텍스트 완전 초기화 — 새 태스크 시작 시 이전 작업의 잔재 제거

/compact

컨텍스트 압축 — 토큰 한계 전 핵심 맥락 보존, 긴 세션 유지

/context

컨텍스트 현황 확인 — 현재 로드된 정보 투명하게 파악

/plan

실행 전 계획 수립 — 대규모 변경 전 접근 방식 사전 검토

/copy

응답 클립보드 복사 — GitHub, Confluence, Notion 등으로 즉시 전달

/export

대화 내용 내보내기 — 히스토리 보존, 팀 공유, 감사 기록 보관

/security-review

보안 검토 — PIPA 준수, 인증·인가 취약점 탐지, 금융·헬스케어 특히 중요

03 중급: 일상 개발 효율을 높이는 8개 커맨드

/resume

이전 세션 재개 — 컨텍스트 재설명 없이 작업 흐름 즉시 복원

/rewind

대화 되감기 — 잘못된 방향으로 흘러간 대화를 분기점으로 되돌리기

/fork

대화 분기 — 동일 컨텍스트에서 여러 접근 방식을 병렬 탐색

/btw

맥락 메모 추가 — 흐름 방해 없이 부가 제약 조건·정보 주입

/add-dir

디렉터리 추가 — 멀티레포·모노레포 구성에서 참조 범위 확장

/tasks

태스크 관리 — 복수 작업의 진행 상황을 한눈에 추적

/desktop

데스크톱 연동 — Claude Code와 GUI 앱 상호작용 활성화

/remote-control

원격 조작 — CI/CD 파이프라인 통합, 서버 자동화 활용

04 하급 및 분석 커맨드

Tier B — 필요할 때 참고하는 설정·기능 커맨드 (20+)

- ▶ /debug — 디버그 모드
- ▶ /privacy-settings — 개인정보 설정
- ▶ /sandbox — 격리 환경 실행 ★
- ▶ /model — AI 모델 선택
- ▶ /mcp — MCP 서버 관리
- ▶ /hooks — 커스텀 훅 설정
- ▶ /config — 전체 설정
- ▶ /ide — IDE 연동
- ▶ /chrome — Chrome 연동
- ▶ /help, /login, /logout ...

분석 커맨드

/usage

전체 사용량 현황 확인

/stats

세션별 통계 데이터

/insights

사용 패턴 분석·개선 제안

기업 환경 활용 예시

- 팀 도입 ROI 측정
- 비용 효율성 검토
- 워크플로우 최적화

상급

컨텍스트 관리 3가지 완전 활용 가이드

/clear(완전초기화), /compact(전략적 압축), /context(사용량 가시화)
컨텍스트를 능동적으로 관리해 Claude Code 응답 품질을 극대화한다

01 왜 컨텍스트 관리가 중요한가?

컨텍스트 미관리 시

- ✕ 이전 태스크 문맥이 섞여 엉뚱한 코드 생성
- ✕ 토큰 포화로 새 파일 읽기 불가
- ✕ 응답 품질 저하 및 일관성 상실
- ✕ 갑작스러운 컨텍스트 한계 도달
- ✕ 세션 폐기 후 처음부터 재설명 필요

3형제로 능동적 관리 시

- ✓ 항상 깨끗한 컨텍스트에서 정확한 답변
- ✓ 중요 문맥 보존하며 용량 77% 회복
- ✓ 응답 품질을 지속적으로 높은 수준 유지
- ✓ 컨텍스트 한계를 예방적으로 관리
- ✓ 세션 연속성 유지로 재설명 비용 제로

02 컨텍스트 관리 3형제 — 커맨드 일람

/clear

대화 히스토리 완전 초기화

핵심 장점: 컨텍스트 오염 방지, 토큰 효율 즉시 회복

사용 시점: 태스크 전환 시, 응답이 이상하다고 느낄 때

/compact [instructions]

대화를 요약·압축하여 용량 확보

핵심 장점: 중요 문맥 보존하면서 최대 77% 용량 회복

사용 시점: 장시간 세션 중 80% 도달 전 선제적 압축

/context

컨텍스트 사용량·구성 수치 가시화

핵심 장점: 데이터 기반으로 최적 타이밍 판단

사용 시점: 수치 확인, 비용 효율적 개발 환경

03 /clear 실전 데모 — 컨텍스트 리셋

상황

결제 기능과 인증 기능을 번갈아 작업한 장시간 세션. 인증 엔드포인트 수정을 요청했더니 결제 기능 코드를 참조한 엉뚱한 답변이 돌아왔다.

Terminal

> /clear

✓ 대화 히스토리가 초기화되었습니다

> "사용자 인증 엔드포인트를 수정해줘"

→ 깨끗한 컨텍스트로 정확한 답변 생성

실무 팁

"응답이 이상하다"고 느낄 때 가장 빠른 해결책. Spring Boot, NestJS 등 여러 모듈을 동시에 다루는 엔터프라이즈 백엔드 개발 환경에서 특히 효과적이다.

04 /compact 실전 데모 — 전략적 컨텍스트 압축

상황

TypeScript 마이그레이션 프로젝트 진행 중 컨텍스트 사용량 82% 도달. 새 파일 읽기 불가 상태이지만 이제까지 쌓은 마이그레이션 방침과 결정 사항은 반드시 보존해야 한다.

Terminal

> /compact "TypeScript 마이그레이션 방침과 결정 사항 보존"

✓ 대화를 압축했습니다

압축 전: 164K tokens → 압축 후: 38K tokens (77% 절감)

보존 항목: TypeScript 마이그레이션 방침, strict 모드 활성화 결정

> /context

컨텍스트 사용량: 19% (38K / 200K tokens)

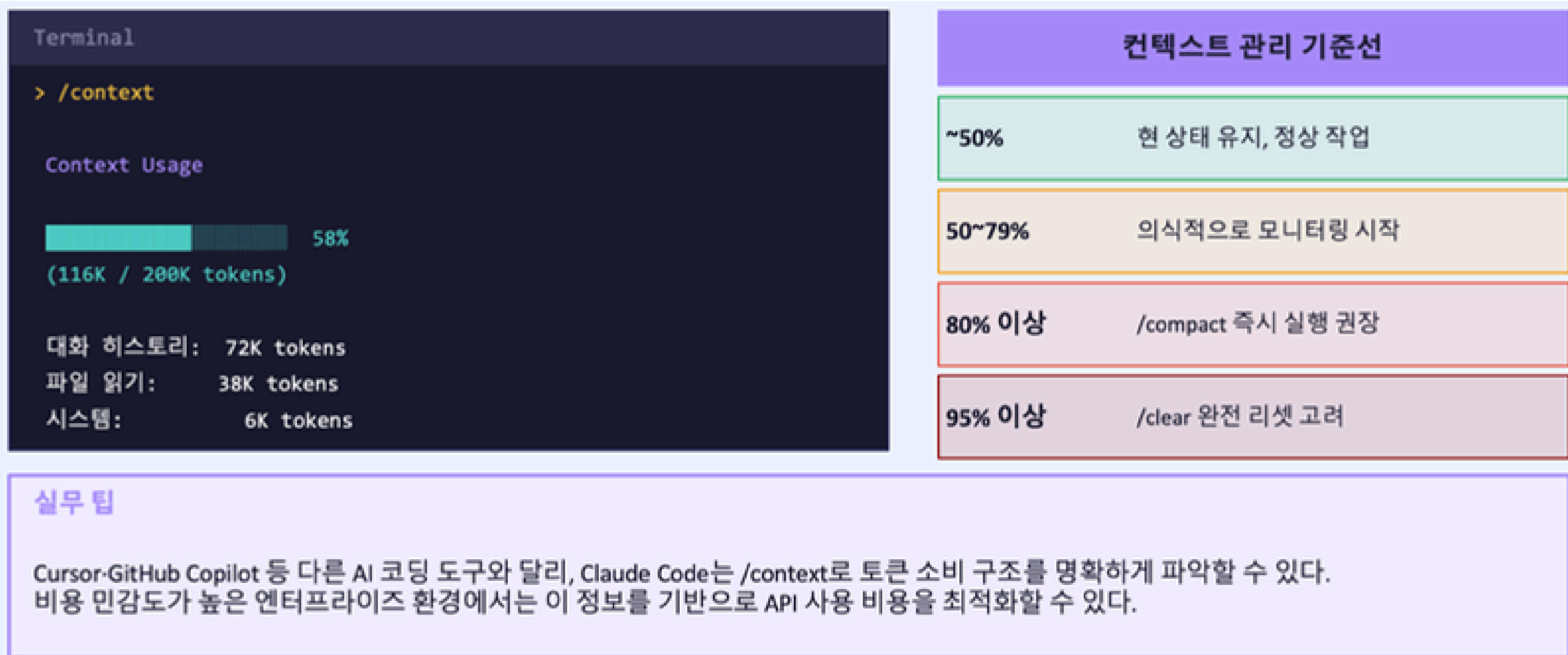
77% 용량 회복

중요 문맥 100% 보존

실무 팁

Jira 스프린트 작업 시 결정 사항을 instructions에 명시하면 세션이 길어져도 핵심 컨텍스트를 유지할 수 있다.

05 /context 실전 데모 — 사용량 가시화



상급

/plan으로 재작업 제로를 달성하는 실전 가이드

/plan 후 구현 시작
✓사전에 설계 검토 완료
✓재작업 제로
✓클린 컨텍스트 보장

01 /plan 커맨드 개요

/plan

플랜 모드로 전환 — 코드 변경 없이 구현 계획만 먼저 수립한다

플랜 확정 후 컨텍스트 자동 초기화 → 탐색 잡음 없이 클린한 구현 단계 진입

핵심 장점

대규모 변경 전 계획 검토·수정 → 재작업 최소화
플랜 확정 후 컨텍스트 자동 초기화

활용 사례

대규모 리팩터링
신기능 구현
아키텍처 의사결정

사용 흐름

/plan → 계획 검토·수정
→ 승인 → 구현 시작
→ 재작업 없음

02 /plan 실전 데모 — JWT → 세션 기반 인증 전환

상황

인증 기능을 JWT에서 세션 기반으로 전환 필요. 영향 범위가 넓어 바로 구현 시 설계 오류를 뒤늦게 발견하고 대규모 재작업이 발생할 위험이 크다.

Terminal

> /plan

✓ 플랜 모드로 전환했습니다

> "인증 기능을 JWT에서 세션 기반으로 전환해줘"

→ 구현 계획 제시 (코드 변경 없음):

1. 세션 스토어 선택 (Redis vs DB)
2. 세션 미들웨어 구현
3. 기존 엔드포인트 마이그레이션 (12곳)
4. JWT 관련 코드 삭제 5. 테스트 업데이트

영향 파일: 8개 | 예상 변경: ~240행 | 리스크: Session Fixation 대응

→ 계획 검토·수정 → 승인 → 컨텍스트 자동 초기화 → 구현 시작

03 /plan의 두 가지 핵심 강점

강점 1 | 재작업을 대폭 줄인다

3개 파일 이상 변경이 예상되는 작업에 /plan을 먼저 실행하면 설계 오류에 의한 재작업을 크게 줄인다.
계획 없이 즉시 구현에 돌입했다가 발생하는 30분 이상의 되돌리기 비용을 방지할 수 있다.

▶ 레거시 → AWS ap-northeast-2 이관

▶ PIPA 준수 세션 관리 재설계

▶ 모놀리식 → 마이크로서비스 분리

강점 2 | 플랜 확정 후 컨텍스트 자동 초기화 — 품질 보호 메커니즘

계획 단계

다양한 파일 탐색
여러 접근 방식 비교
→ 컨텍스트에 잡음 누적



자동 초기화

플랜 확정 시점에
컨텍스트 클리어
→ 잡음 완전 제거



구현 단계

확정 계획만 기반으로
클린 컨텍스트 시작
→ 정확한 코드 생성

04 /plan을 사용해야 하는 상황

✓ 3개 파일 이상

변경이 예상되는 작업은 무조건 /plan 먼저

✓ 아키텍처 레벨 판단

hexagon·클린 아키텍처, 레이어 설계, 의존성 방향 결정 시

✓ 여러 접근 방식 비교

Redis vs DB, REST vs GraphQL 등 기술 선택 검토 시

✓ 대규모 리팩터링·신기능

스프린트 단위 대형 태스크 착수 전

✓ 팀 설계 문서 초안

/copy와 조합하면 Confluence·Jira에 즉시 붙여넣기 가능

/plan의 출력 결과는 PR 설명문·Confluence 기술 설계 문서 초안으로 바로 활용 가능

/plan → 계획 검토·수정 → 승인 (컨텍스트 자동 초기화) → 구현 시작 → 재작업 없이 완료

상급

결과물 공유 · 기록 · 보안 검토 완전 가이드



/copy (포맷손상없는 응답공유)
/export (설계논의를 자산으로 기록)
/security-review (PR전 보안자동검증)

01 3개 커맨드 일람 — 역할과 활용 사례

`/copy`

마지막 응답을 클립보드에 복사

핵심 장점: 포맷 손상 없이 Slack·GitHub·Confluence에 즉시 전달

사용 시점: 코드·답변 공유, PR 코멘트 작성 시

`/export [file]`

대화 전체를 Markdown 파일로 내보내기

핵심 장점: 설계 의사결정 경위를 팀 지식 자산으로 영구 보존

사용 시점: 설계 논의 기록, ISMS 감사 대응, 온보딩 문서화

`/security-review`

코드 내 보안 취약점 자동 분석

핵심 장점: SQL Injection·XSS·시크릿 노출 등 자동 검출로 리뷰어 부담 경감

사용 시점: PR 머지 전 필수 실행, PIPA·금융보안원 규정 대응

02 /copy 실전 데모 — 포맷 손상 없는 응답 공유

마우스 드래그 복사 시

1 에러 원인: undefined is not
2 a function at line...
3 [불필요한 줄번호 혼입]
→ 포맷 완전 붕괴

VS

/copy 실행 시

✓ 클립보드에 복사 (1,247자)
에러 원인: undefined is
not a function at...
→ 완벽한 포맷 유지

Terminal

> /copy

✓ 클립보드에 복사했습니다 (1,247자) → Slack, PR 코멘트, Notion에 그대로 붙여넣기

실무 팁

▶ GitHub PR

▶ Slack 채널

▶ Confluence

▶ Jira 티켓

▶ Notion 페이지

/plan 계획서 · /export 설계 문서와 조합해 Claude Code 결과물을 외부 도구로 원활하게 이동

03 /export 실전 데모 — 설계 논의를 팀 지식 자산으로

상황

인증 설계 장시간 논의 완료 — JWT 페지 이유, 세션 방식 선정 근거, Redis 도입 결정이 대화 속에 흩어져 있다. 팀 지식 자산으로 보존이 필요하다.

Terminal

```
> /export design-decision-auth.md
```

```
✓ 대화를 내보냈습니다: design-decision-auth.md | 메시지 수: 24 | 파일 크기: 15.2KB
```

한국 기업 환경에서의 활용 포인트



ISMS 감사 대응

설계 의사결정 근거 문서화로
감사 추적(Audit Trail) 요건 충족



PIPA 준수 검토

개인정보 처리 방침 논의 보존으로
책임 소재 명확화



SI 산출물 관리

발주사 제출용 기술 검토
문서 초안으로 활용



신규 입사자 온보딩

설계 결정 배경을 빠르게
파악하는 지식 자산

04 /security-review 실전 데모 — PR 전 자동 보안 스캔

Terminal

```
> /security-review
🔍 보안 스캔 실행 중...

Security Review Results

● Critical: 1
  SQL Injection (users.ts:42)

● Warning: 2
  XSS risk (Comment.tsx:18)
  Hardcoded secret (config.ts:7)

● Info: 3 (rate limit, CSRF, CORS)
```

심각도별 대응 방법

● Critical

PR 머지 전 반드시 수정

● Warning

트리아지 후 수정 또는
예외 처리 문서화

● Info

백로그 등록 또는
기술 부채로 관리

한국 규정 준수 관련성

- ▶ PIPA(개인정보보호법): SQL Injection·인증 바이패스 → 과징금·형사처벌 대상
- ▶ 금융보안원 전자금융 기반시설 취약점 점검: 핀테크·뱅킹 서비스 정기 점검 사전 대응
- ▶ ISMS-P 인증: 보안 취약점 관리 프로세스 증빙 자료로 활용 가능

중급

세션 관리 3가지로 다음 날도 막힘없이 재개하는 방법



- /resume (전날 세션 즉시복구)
- /rewind (코드+대화 동시 되감기)
- /fork (A안·B안 병렬 탐색)

01 세션 관리 3가지 — 커맨드 일람

/resume [session]

과거 세션 불러와 즉시 재개

핵심 장점: 10분+ 컨텍스트 재구축 → 커맨드 1번으로 완료

사용 시점: 날을 넘기는 개발, 크래시 긴급 복구, 중단 작업 재개

/rewind

코드+대화를 선택한 과거 시점으로 동시 복구

핵심 장점: git reset보다 안전·빠름. 수동 복구 20분 → 1분

사용 시점: Claude 제안이 의도와 다를 때, 실험적 변경 철회

/fork [name]

현재 대화를 분기해 독립 세션 생성

핵심 장점: A안·B안 병렬 탐색. 실패해도 원본 세션 완전 보존

사용 시점: 설계 방향 고민 시, 기술 스택·아키텍처 비교

02 /resume 실전 데모 — 전날 세션 즉시 복구

/resume 없이 다음 날 출근

- × 어디까지 했더라? (5분 기억 복원)
- × 컨텍스트 재설명 (10분+)
- × 크래시 복구 불가
- = 총 15분+ 손실 후 실제 작업 시작

VS

/resume 실행 후

- ✓ 세션 목록에서 선택 (30초)
- ✓ 컨텍스트 전체 자동 복원
- ✓ 크래시 세션도 즉시 복구
- = 30초 만에 실제 작업 시작

Terminal

> /resume

1. auth-feature-day1 (2시간 전) – JWT 마이그레이션
2. bugfix-payment (어제) – 테스트 전체 통과

> 1

✓ 세션 'auth-feature-day1' 재개 완료 → 즉시 작업 시작

실무 팁

스프린트 기간 중 매일 이어지는 작업이나 긴급 버그 대응 후 원래 작업 복귀 시 특히 효과적.
세션 이름을 Jira 티켓 번호나 기능 브랜치명과 일치시키면 팀 내 작업 추적도 용이해진다.

03 /rewind 실전 데모 — 코드+대화 동시 되감기

상황

컴포넌트 리팩터링 요청 → 5개 파일 변경됐지만 useReducer가 아닌 useState로 구현됨. git stash+diff 수동 복구는 20분 소요.

Terminal

> /rewind

1. [직전] Hooks 리팩터링 완료

2. [3분 전] 리팩터링 시작 전 + [여기로](#)

3. [8분 전] 컴포넌트 구성 논의

> 2

✓ '리팩터링 시작 전' 상태로 복원 (5개 파일 복구)

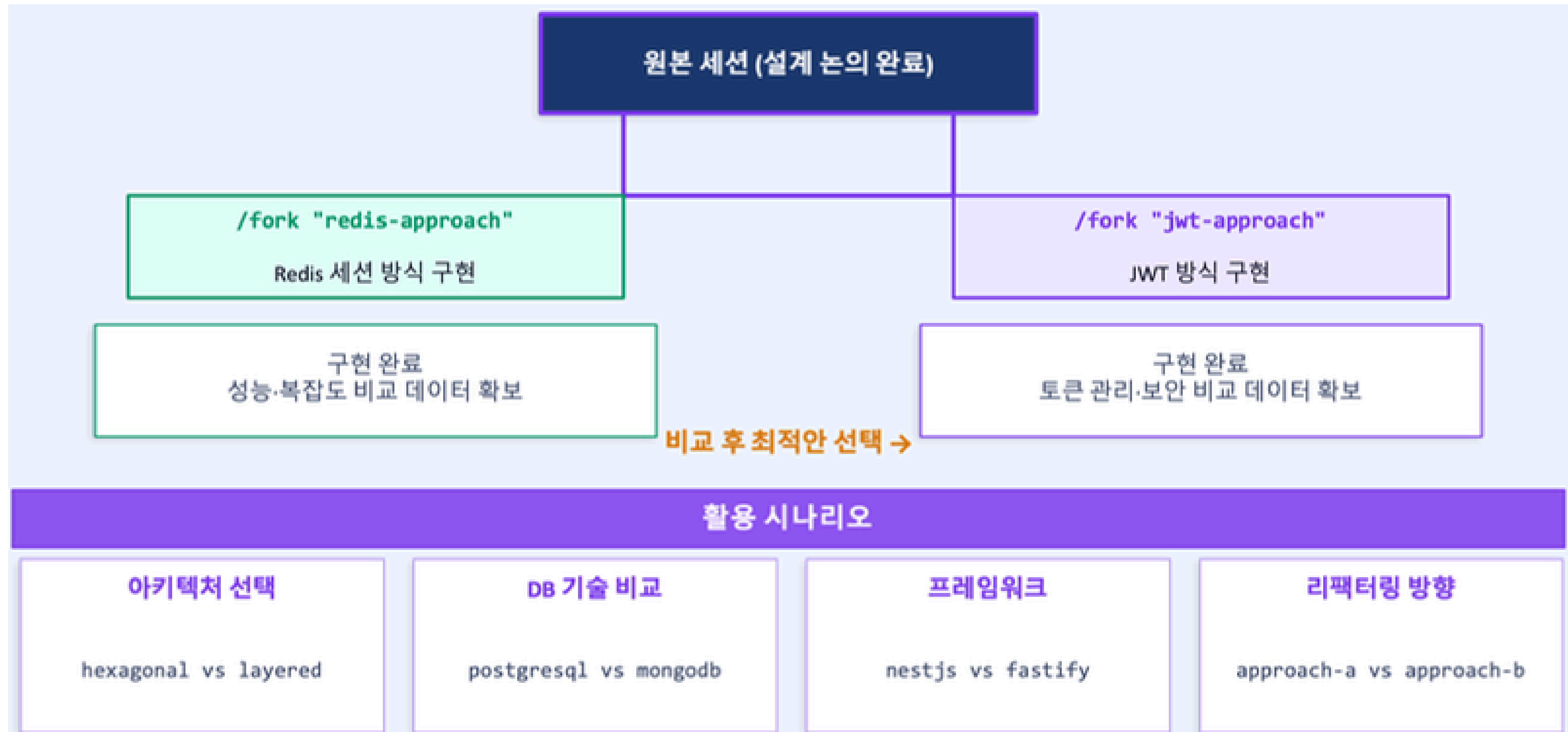
> "useReducer 패턴으로 리팩터링해줘"

+ 의도대로 구현 완료

git reset vs /rewind 비교

비교 항목	git reset	/rewind
코드 변경 복구	✓	✓
대화 컨텍스트 복구	✗	✓
조작 난이도	중간 (커밋 해시 필요)	낮음 (목록 선택)
소요 시간	~20분	~1분

04 /fork 실전 데모 — A안·B안 병렬 탐색



중급

작업 흐름을 끊지 않고 개발 효율을 높이는 3가지



/btw (서브 질문 처리, 메인 작업 자동 복귀)
/add-dir (모노레포 횡단, 단일 세션 수정)
/tasks (백그라운드 실행, 대기 시간 제로)

01 중급 커맨드 일람

/btw

메인 태스크 중단 없이 서브 질문 처리

핵심 장점: 문맥 보존 → 자동 복귀. 재설명 5분 손실이 제로

사용 시점: 구현 중 빠른 기술 확인, 짧은 서브 질문

/add-dir

추가 디렉터리를 현재 세션에 통합

핵심 장점: 모노레포·멀티레포 횡단 수정 단일 세션으로 처리. 세션 재시작 불필요

사용 시점: 공유 타입 동기화, 멀티 패키지 동시 수정, SI 멀티 모듈

/tasks

백그라운드 태스크 실행·진행 현황 추적

핵심 장점: 테스트·빌드 대기 시간 제로. 복수 태스크 병렬 관리

사용 시점: 장시간 테스트, 프로덕션 빌드, Docker 이미지 빌드

02 /btw 실전 데모 — 메인 작업 문맥 보존 서브 질문

일반 질문 시

> "TypeScript never 언제 써?"

- Claude Code: TypeScript 해설 모드 진입
- 메인 작업 (에러 핸들링) 문맥 소실
- 원래 작업 복귀 위해 재설명 필요
- = 재설명 5분+ 낭비

VS

/btw 사용 시

> /btw "TypeScript never 언제 써?"

- 간결 답변: exhaustive check 패턴
switch default에 never → 미처리 케이스
컴파일 시점 검출 가능
- 자동 복귀: 에러 핸들링 구현 계속
- = 재설명 0분

/btw 최적 활용 상황

NestJS 데코레이터 동작 방식

Spring Boot 어노테이션 순서

TypeScript 타입 추론 규칙

React 훅 의존성 배열 규칙

SQL 쿼리 최적화 힌트

실무 팁

/btw는 짧고 명확한 기술적 확인에 최적. 긴 설명이 필요한 주제는 /fork로 별도 탐색하는 것을 권장.
"잠깐 확인하고 싶다"는 순간에만 사용하는 것이 핵심이다.

03 /add-dir 실전 데모 — 모노레포 횡단 수정 단일 세션

상황: 모노레포에서 shared-types의 User 타입에 새 필드 추가 → backend + frontend 동시 업데이트 필요

Terminal

```
> /add-dir ../frontend
```

```
✓ 디렉터리 추가: /projects/frontend
```

```
> /add-dir ../shared-types
```

```
✓ 디렉터리 추가: /projects/shared-types
```

```
> "User 타입에 phoneNumber 추가하고 backend·frontend 함께 업데이트해줘"
```

```
→ shared-types/src/user.ts / backend/src/api/users.ts / frontend/src/hooks/useUser.ts 동시 수정
```

한국 기업 환경 활용 포인트

Nx-Turborepo 모노레포

Next.js+NestJS+공유 패키지
단일 세션 횡단 수정

멀티레포 마이크로서비스

서비스 간 공유 타입·API 계약
일관성 있는 변경

Maven/Gradle 멀티 모듈

common/domain 변경 후
연관 모듈 함께 업데이트

04 /tasks 실전 데모 — 대기 시간을 생산 시간으로

테스트 실행 중 (대기 방식)	VS	Ctrl+B → /tasks (병렬 방식)	
- 테스트 실행 중... (5분) - 멍하니 대기 - 멍하니 대기 - 멍하니 대기 = 5분 전체 낭비		#1 테스트 (백그라운드 실행 중) ✓ #2 lint 수정 (완료) ✓ #3 타입 체크 (완료) + 신규 기능 구현 진행 중 = 5분 동안 3개 작업 완료	
Terminal <pre> > "테스트 스위트 전체 실행해줘" (Ctrl+B → 백그라운드 전환) > /tasks #1 테스트 스위트 전체 실행 (실행 중 2:34) ✓ #2 lint 수정 (완료 1분 전) ✓ #3 타입 체크 (완료 3분 전) </pre> → 테스트 완료 앞까지 대기 못 — 그 사이 다른 작업 진행 못			
활용 시나리오 — 백그라운드 태스크별 병렬 작업			
BG: Jest/Vitest 전체 테스트 → 신규 기능 구현·문서 작성	BG: Next.js 프로덕션 빌드 → PR 설명문·코드 리뷰	BG: Docker 이미지 빌드 → Helm 차트·k8s 매니페스트	BG: ESLint 전체 스캔 → TypeScript 타입 오류 수정

중급

장소와 디바이스를 초월하는 개발 경험



/desktop (CLI → 데스크톱 앱 심리스 전환)
/remote-control (스마트폰에서 로컬 세션 조작)

01 중급 커맨드 일람

/desktop

CLI 세션을 데스크톱 앱으로 심리스 전환

핵심 장점: 세션 연속성 유지. Markdown 렌더링·이미지·스크롤 쾌적

사용 시점: 장문 응답, UI 버그 확인, Markdown 문서 검토

/remote-control

로컬 PC 세션을 스마트폰·브라우저에서 원격 조작

핵심 장점: 30초 셋업. 코드 로컬 보존. MCP·커스텀 툴 그대로 작동

사용 시점: 외출 중 장시간 태스크 모니터링, 권한 승인 원격 대응

02 /desktop 실전 데모 — CLI에서 데스크톱 앱으로

Terminal

> /desktop
✓ 데스크톱 앱에서 세션을 엽니다...

→

Claude Code Desktop

동일 세션 → 완전 렌더링 UI

CLI vs 데스크톱 앱 비교		
기능	CLI (터미널)	데스크톱 앱
장문 응답 스크롤	불편	쾌적
Markdown 렌더링	미지원 (원문)	완전 렌더링
이미지·다이어그램	미지원	지원
코드 하이라이팅	제한적	완전 지원
복사·공유	마우스 드래그	버튼 클릭

사용 팁

/desktop은 단방향 전환이다. 데스크톱 앱으로 이동 후에는 앱에서 계속 작업하거나 새 터미널 세션을 시작해야 한다. 장문 응답과 Markdown 문서가 많아지기 시작하는 순간이 전환 타이밍이다.

03 /remote-control 실전 데모 — 스마트폰에서 로컬 세션 조작

상황

PC에서 대규모 리팩터링을 Claude Code에게 맡기고 외출. 도중 권한 승인 요청이 오면 스마트폰에서 즉시 처리하고 싶다. 기존 방식 (SSH+tmux)은 30분+ 셋업 필요.

Terminal

```
> /remote-control
```

Remote Control

URL: <https://cc.anthropic.com/r/xxx>

QR: [스페이스바로 QR 코드 표시]

이 URL로 다른 디바이스에서 조작 가능

종료: /remote-control 재실행

claude.ai/code

auth-refactor-sprint3

권한 승인 대기 중...

승인 ✓

추가 지시 입력...

셋업 시간 비교

SSH + tmux + 포트포워딩

셋업: 30분~1시간 | 코드: 로컬 유지

클라우드 IDE (GitPod 등)

셋업: 5~10분 | 코드: 클라우드 이전

/remote-control

셋업: 30초 | 코드: 로컬 완전 보존

04 /remote-control 핵심 강점 & 실전 활용 시나리오

코드·파일 로컬 완전 보존

클라우드로 전송되는 건 대화 메시지뿐.
코드·환경변수·API 키·MCP 설정 모두 PC에 유지.
금융·의료·공공 분야에서도 안전하게 활용 가능

로컬 환경 설정 그대로 유지

원격 접속 중에도 MCP 서버·커스텀 툴·
프로젝트 권한 설정이 모두 작동.
기능 제한 없는 완전한 세션 조작

Tips: 더 편하게 쓰는 법

단축키: /rc (새 세션: claude rc)
/rename 후 /rc → 모바일 식별 용이
/config: 모든 세션 자동 원격 활성화

실전 활용 시나리오



출근 전

/rename → /remote-control
작업 실행 후 출발



이동 중

스마트폰으로 접속
진행 상황 확인·권한 승인



회의 중

추가 지시 입력
(자연어로)



복귀 후

PC에서 결과 확인
/security-review → PR

하급

알아두면 유용한 커맨드 완전 일람



디버그/진단: `/debug` · `/status`

설정/개인화: `/config` · `/privacy` · `/keybindings`

모델/성능: `/model` · `/fast`

확장/통합: `/mcp` · `/hooks` · `/chrome` · `/ide`

01 디버그·진단 계열 & 설정·개인화 계열

🔧 디버그·진단 계열

/debug

세션 로그 분석·ON/OFF 전환
문제 원인 신속 특정

/status

버전 정보·접속 상태 표시
API 서버 vs 로컬 문제 구분

사용 패턴

/status → 접속 상태·버전 확인 (30초)

/ debug → 세션 로그 상세 분석

⚙️ 설정·개인화 계열

/config

GUI처럼 전체 설정 안전 열람·변경

/privacy-settings

모델 학습 데이터 활용 허가 제어
⚠️ PIPA 적용 프로젝트 시 반드시 확인

/terminal-setup

iTerm2·Terminal.app 키바인드 최적화
초기 설치 시 1회 실행

/keybindings

키바인드 확인·커스터마이징

/statusline

상태 표시줄 최소화 → 집중 모드

02 AI 모델·성능 계열 & 확장·통합 계열

🎁 AI 모델·성능 계열

/model [model]

모델 전환으로 비용·품질 최적화
인수 없이 실행 → 사용 가능 모델 목록

/fast [on|off]

패스트 모드로 대기 시간 단축
탐색 단계에서 ON, 정밀 작업에서 OFF

비용 최적화 패턴

단순 작업 → /model sonnet + /fast on

복잡 작업 → /model opus + /fast off

(리팩터링·설계 검토 등)

🔌 확장·통합 계열

/mcp

GitHub·Slack·Jira·DB 직접 조작

/hooks

저장 시 자동 테스트·포맷 자동화

/chrome

브라우저 화면 직접 확인·조작

/ide

VS Code·IntelliJ 심리스 연동

/plugin

서드파티 플러그인 설치·관리

/reload-plugins

플러그인 재시작 없이 즉시 재로드

/agents

서브에이전트 목록·생성·편집

03 정보·인증 계열 & 중급 전체 커맨드 요약

📁 정보·인증 계열	하급 전체 커맨드 (20+) 빠른 참조	
/skills 사용 가능 Skills 목록 확인	/debug	/status
/release-notes 최신 업데이트·변경 이력 확인	/config	/privacy-settings
/help 커맨드 목록·사용법 빠른 확인	/terminal-setup	/keybindings
/login 세션 만료 시 CU 내 즉시 재인증	/statusline	/model
/logout 공용 환경 작업 후 계정 안전 분리	/fast	/mcp
보안 팁 공용 개발 서버·페어 프로그래밍 환경 사용 후 /logout 필수. OAuth 토큰 잔존 시 타인이 계정 권한으로 사용 가능.	/hooks	/chrome
	/ide	/plugin
	/reload-plugins	/agents
	/skills	/release-notes
	/help	/login
	/logout	/sandbox ⭐

하급

야간 자동 파이프라인 구축 완전 가이드



/sandbox (OS 레벨 격리 승인 84% 절감)

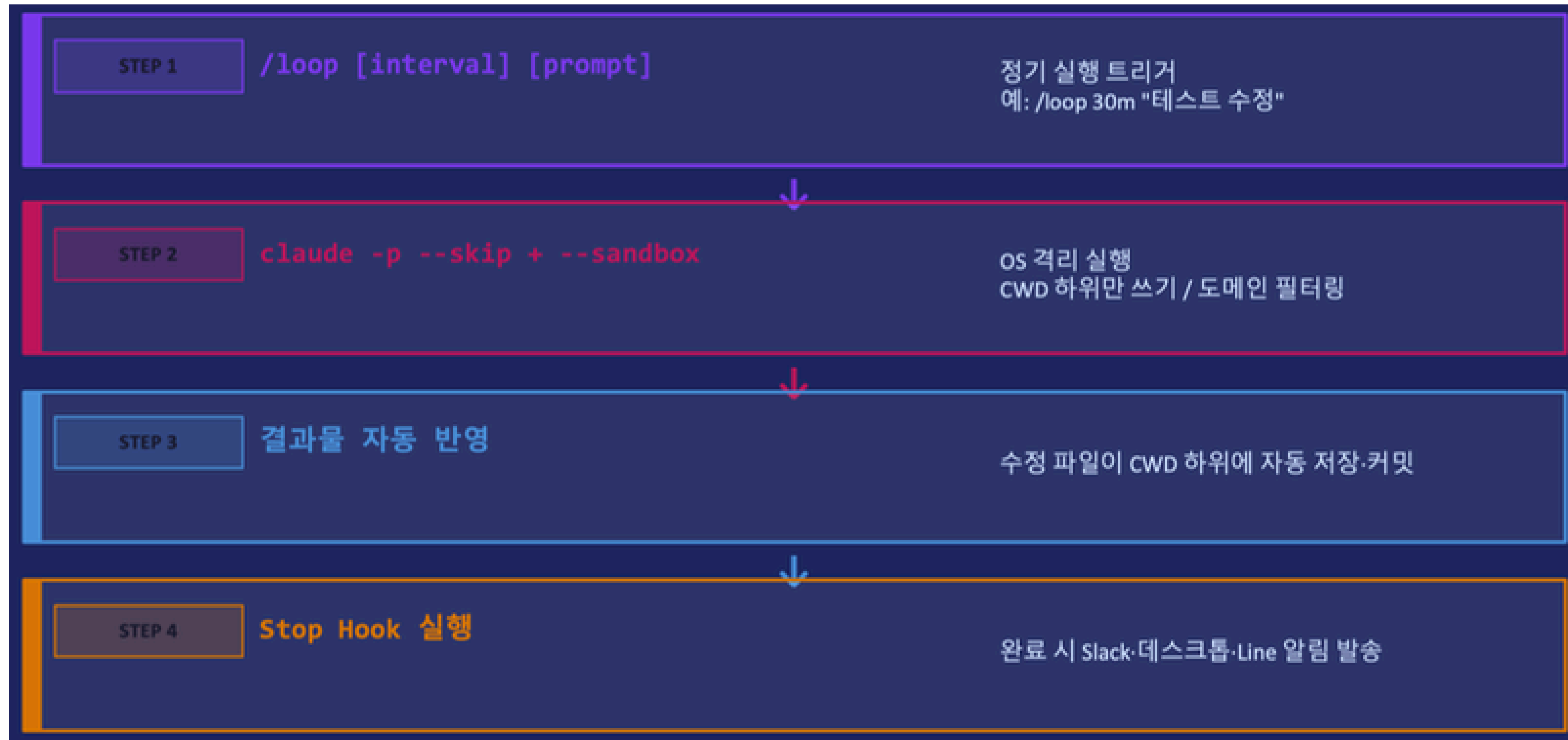
/loop (정기 자동 실행, 인터벌 지정)

Hooks (완료 알림, Slack·데스크톱)

01 왜 /sandbox가 필요한가 — 안전한 무인 자동화의 핵심

--dangerously-skip-permissions 단독		+ --sandbox 병용
✕ 파일 시스템 전체 무제한 쓰기 ✕ 네트워크 접근 전체 무제한 ✕ 중요 파일 실수 덮어쓰기 위험 ✕ 의도치 않은 외부 API 호출 ✕ 보안 정책 위반 가능성 ✕ 승인 0% (전부 자동 스킵)	→	✓ CWD 하위만 쓰기 허용 (OS 격리) ✓ 네트워크 도메인 필터링 적용 ✓ 중요 파일 보호 ✓ 의도치 않은 외부 접근 차단 ✓ PIPA·보안 정책 준수 가능 ✓ 승인 프롬프트 약 84% 절감
결론: 무인 자동화에 단독 사용은 위험 — 전체 무제한 접근		결론: OS 레벨 격리 + 84% 승인 절감 → 안전한 무인 자동화 실현

02 /sandbox + /loop + Hooks 자동 파이프라인 아키텍처



03 실전 자동화 파이프라인 예시 6선

🔪 테스트 자동 수정

간격: 30분

실패 케이스 수정 → 재실행 → 전 통과

✨ 코드 품질 감시

간격: 1시간

ESLint-Prettier 경고 파일 수정 → 커밋

🔒 보안 취약점 스캔

간격: 4시간

npm audit → critical/high 패치 적용

📦 의존성 정기 업데이트

간격: 6시간

npm outdated 확인 → 마이너 업데이트 → 테스트

📄 API 문서 자동 생성

간격: 2시간

변경 엔드포인트 Swagger 자동 업데이트

🚀 배포 후 헬스 체크

간격: 5분

프로덕션 헬스체크 → 이상 시 즉시 Slack

04 완료 알림 설정 & 주의사항

완료 알림 설정 방식

macOS

```
osascript -e 'display notification "완료" with title "Claude Code"'
```

Linux (WSL2)

```
notify-send "Claude Code" "태스크 완료"
```

Slack

```
curl -X POST --data '{"text":"완료"}' $SLACK_WEBHOOK_URL
```

주의사항

🕒 세션 범위

/loop는 세션 내에서만 동작
터미널 종료 시 자동 정지

📅 자동 만료

3일 후 자동으로 기한 만료

⚡ 동시 실행

최대 50개의 동시 태스크

🔄 영구 스케줄링

지속 실행: Desktop Scheduled Tasks
또는 GitHub Actions 활용 권장

📁 추가 경로

sandbox.filesystem.allowWrite로
CWD 외부 경로 쓰기 허용 가능

05 분석 커맨드 — 비용 관리·워크플로우 개선 모니터링

/usage

플랜 5시간·주간 사용 한도 및 잔여량 표시

개인: 한도 도달을 사전 감지 → 작업 중단 방지

팀·기업: 스프린트 기간 팀 전체 사용량 추적 → 월말 예산 초과 방지

/stats

모델별 토큰·비용·세션 수 통계 시각화

개인: 비용 트렌드 파악 → 장기 계획·모델 최적화 근거

팀·기업: Opus vs Sonnet 비용 비교 → 태스크별 최적 모델 선택

/insights

세션 성공률·생산성 지표·사용 패턴 분석

개인: 워크플로우 병목 발견 → 개발 방식 객관적 개선

팀·기업: 팀원별 생산성 지표 → 온보딩 효과 측정·표준화 기준

탭 내비게이션: /config ↔ /status ↔ /usage — 실행 후 탭 전환으로 설정·접속 상태·사용량을 한 화면에서 점검

06 권장 6단계 학습 로드맵

1단계 이번 주	<code>/clear + /context</code>	컨텍스트 상태 파악과 초기화 습관
2단계 다음 주	<code>/compact</code>	문맥 보존 압축 체험
3단계 이번 달	<code>/plan + /security-review</code>	구현 전 계획·PR 보안 검토 습관
4단계 다음 달	<code>/resume + /rewind</code>	세션 관리와 안전한 실험 습관
5단계 2개월 차	<code>/btw + /fork + /add-dir</code>	작업 흐름 마찰 제거
6단계 3개월 차	<code>/sandbox + /loop + Hooks</code>	야간 자동화 파이프라인 구축
한 단계씩 실제 작업에 적용하면서 체감 효과를 확인하다 보면 자연스럽게 전체를 사용할 수 있게 된다		